

HANDS-ON EXERCISE

Insurance Document Intelligence Pipeline

India — Batch 1 — Claims Operations & Settlement

Week 4 · Day 3 · Self-Paced Lab, Step by Step · Prepared by Rajesh Pasham

Before You Begin

This lab builds directly on three things you already have: your Week 1 Claims Operations Ontology, your Week 2 ClaimsCopilot, and your Week 3 ClaimApprovalReasoning workflow. Today adds the thing none of them could do: actually read what's inside a claim document, not just check whether one exists.

Every click in this document uses the exact tool names, field labels, and buttons confirmed against Palantir's own platform walkthroughs — not paraphrased or guessed. Today also introduces one new tool, Pipeline Builder, which you have not used before in this program; every step inside it is explained from first principles.

Read This First: Your Sample Documents Are Provided

Platform Mechanic

Five real, ready-to-use PDF files are provided alongside this document, built from three actual claims already in your dataset. Part 1 has you confirm them, so Parts 2 onward can focus entirely on the pipeline itself.

A Second Thing to Read First: The Pipeline, End to End

Stage	What Happens
Upload (Part 2)	Your sample documents become a media set — a governed, schema-aware file collection, from Day 1
Extract (Part 3)	The Extract text from PDF transform reads the content out of each document, followed by three cleanup transforms that turn it into one clean row per page
Chunk & Identify (Part 4)	Extracted text is split into pieces, and each piece is given a unique, traceable identifier
Embed (Part 5)	Each chunk is converted into a vector using the Text to embeddings transform
Link (Part 6)	Each chunk becomes an Ontology object, linked back to its Claim — the same Link Type discipline from Week 1
Search & Use (Parts 7–8)	Your ClaimsCopilot uses a real Semantic search block to search document content by meaning, not just keywords

How Long This Takes

Budget 4–5 hours. This is the first lab to use Pipeline Builder, in addition to the Ontology Manager, AIP Logic, and Object Explorer skills from Weeks 1–3.

What You Need Before Starting

- Your individual Palantir Developer Tier account, already signed in
- Your Week 1 Claims Operations Ontology, including the Claim and Claim Document object types
- Your Week 2 ClaimsCopilot function, published and working
- The five provided sample PDF files, attached alongside this document

Getting Unstuck Without the Trainer

- Re-read the relevant step first — most issues are a missed sub-step, not a platform problem
- Check the Troubleshooting table at the end of this document for the specific symptom you're seeing
- Search the exact error message in the Palantir Developer Community (community.palantir.com)
- Note the blocker in your validation checklist and bring it to Day 4

Lab Outcome

By the end of this lab, you will have a working document intelligence pipeline: a media set of real claim documents, extracted and chunked text, embedded chunks linked back to their Claim objects, and a ClaimsCopilot that uses a real semantic search step to ground its answers in the actual content of those documents — not just their existence.

Orienting Yourself: Four Screens Today

Screen	What It's For
Ontology Manager	Where you'll create the new Claim Document Chunk object type and its Link Type back to Claim. Part 6 happens here.
A Foundry Project Folder	Where you create your media set directly, by uploading files. Parts 1–2 happen here.
Pipeline Builder	New today. Where you extract text, chunk it, and generate embeddings, all as connected transform blocks. Parts 3–5 happen here.
AIP Logic	Where you extend your ClaimsCopilot with a real semantic search step. Parts 7–9 happen here.

Lab Roadmap

Part	Task
1	Confirm your sample claim documents
2	Upload your documents as a media set
3	Extract text from your documents
4	Chunk the extracted text and build a chunk identifier
5	Generate embeddings for each chunk and publish the output
6	Create your Claim Document Chunk object type and link it to Claim
7	Search your chunks by content

Part	Task
8	Add a semantic search step to your ClaimsCopilot
9	Test and verify end to end
10	Create an evaluation suite and publish

Part 1 — Confirm Your Sample Claim Documents

Five PDF files are provided alongside this document, built from three real claims already in your Batch 1 dataset. Take a moment to open each one and connect it to the underlying claim before moving on — you'll be referencing these specific claims and their content throughout the rest of this lab.

Step 1: Locate the five provided files

1. Confirm you have all five files: claim_CLM014_form.pdf, claim_CLM013_form.pdf, claim_CLM013_repair_estimate.pdf, claim_CLM064_form.pdf, and claim_CLM064_police_report.pdf.

Step 2: Open each one and note what it tells you

File	What It Covers	Real claimId
claim_CLM014_form.pdf	A straightforward auto claim — a rear-end collision with minor, uncontested damage	CLM014
claim_CLM013_form.pdf	A High-Value home claim — a kitchen fire with a fire department note recommending a structural assessment	CLM013
claim_CLM013_repair_estimate.pdf	An itemized contractor estimate for the same fire, including a specific structural-repair line item	CLM013
claim_CLM064_form.pdf	A home burglary claim with an inconsistency worth watching: the policyholder reports forced entry	CLM064
claim_CLM064_police_report.pdf	The responding officer's report for the same claim, noting no physical evidence of forced entry was found	CLM064

Step 3: Verify these match your actual dataset

2. Open your claims.csv or your Ontology's Claim objects and confirm CLM014, CLM013, and CLM064 exist, with claimAmount, status, and fraudScore values consistent with what's described in the documents above.

Takeaway

CLM064 is deliberately the more interesting case: the claim form describes a forced-entry break-in, but the police report notes no pry marks or broken glass were found, and the window was unlocked. This is exactly the kind of discrepancy a real SIU investigator would flag — and exactly what should surface later when your ClaimsCopilot searches this claim's real document content in Part 8.

Checkpoint — Confirm Before Continuing

You have all five PDF files, have opened each one, and can identify which real claimId each belongs to.

Part 2 — Upload Your Documents as a Media Set

Step 1: Start the upload

3. Navigate to the Project folder containing your Week 1 Ontology and Week 2/3 functions.
4. Click New in the top right corner (or right-click inside the folder and pick New).
5. Click Upload files...

Platform Mechanic

You can also import files by dragging them directly into your folder.

Step 2: Select your files

6. Click choose from your computer.
7. Select all five PDF files provided alongside this document.
8. Make sure Upload to a new media set is selected — media sets allow you to interact with files in media format, which is what today's pipeline needs, rather than rows in an ordinary dataset.
9. Click Next...

Step 3: Finish the upload

10. Select Transactionless as the write mode.
11. Click Upload.

Step 4: Rename your media set

12. Right-click on your new media set.
13. Rename it to ClaimDocuments.

Step 5: Preview your media set

14. Double-click your media set to preview the documents you uploaded.
15. Click on one of the media items to review the full document and confirm it looks correct.

Checkpoint — Confirm Before Continuing

Your ClaimDocuments media set contains all five uploaded PDFs, and previewing one shows the actual document content.

Part 3 — Extract Text from Your Documents

This Part happens in Pipeline Builder, a tool you have not used before in this program. A pipeline here is a graph of connected transform nodes, each one holding a chain of transform blocks — similar in spirit to AIP Logic's blocks, but built for data transformation rather than reasoning.

Step 1: Create a new pipeline

16. Navigate to your Project folder.
17. Right-click inside the folder and pick New (or click New in the top right corner).
18. Select Pipeline Builder from the dropdown.
19. Name your pipeline ClaimDocumentIntelligencePipeline.
20. Select the pipeline style as Batch pipeline.

21. Click Create pipeline.

Step 2: Add your media set as data

22. Click Add Foundry data — you'll find this in the top left of the Pipeline Builder screen.

23. Add your ClaimDocuments media set to your selection using the + button.

24. Click Add data in the bottom right to confirm.

Step 3: Start a new transform node

25. Back on the main Pipeline Builder graph, select the new ClaimDocuments node.

26. Click the Transform button. This creates a transform node, where transformation steps can be chained and a resulting dataset previewed.

27. Rename the new transform node to Process Documents in the top left corner.

Platform Mechanic

A preview of your current transformation's output appears at the bottom of the page, updating each time you add a block. You chain transform blocks by searching for them in the Search transforms and columns... bar at the bottom of the current chain — keep adding blocks one below the other until instructed to finish.

Step 4: Add the Extract text from PDF block

28. In the search bar, search for and add an Extract text from PDF transform block.

29. Select the mediaReference column in the Media reference dropdown.

30. Set the output column name to content.

31. Click Apply.

Platform Mechanic

The Extract text from PDF block extracts text elements from the documents in your media set. In some cases the text is stored as an image — for example, a photo-scanned document — in which case you should change the Extract Method to OCR. All five of today's provided documents are born-digital PDFs, so the default extraction method is correct here.

Step 5: Add the Explode Array with Position block

32. Search for and add an Explode Array with Position transform block.

33. Set the content column as the array to explode.

34. Set the output column name to content, replacing the existing column.

35. Click Apply.

Platform Mechanic

For each row, this block takes the input array and creates a new row for each element — in this case, one row per page. Each new row is identical to the original, except the output column now contains one page's content and its position (page index) as a struct.

Step 6: Add the Extract Many Struct Fields block

36. Search for and add an Extract Many Struct Fields transform block.

37. Set the content column as the struct to process.

38. Set the first locator to position, extracting that property into a new column called pageNumber.

39. Click Add item.

40. Set the second locator to element, extracting that property into the existing content column.

41. Click Apply.

Common Mistake

It's easy to extract only one of the two locators and move on. You need both: pageNumber (from position) as a new column, and content (from element) replacing the existing struct column — otherwise your next steps won't have plain text to work with.

Step 7: Add the Drop Columns block

42. Search for and add a Drop Columns transform block.

43. Select the timestamp column to drop — this column is added automatically when files are converted into rows, and isn't needed for this pipeline.

44. Click Apply.

Step 8: Finish this transformation

45. Click Close in the top right corner.

Checkpoint — Confirm Before Continuing

Looking at the Selection preview at the bottom of your screen, you should see one row per page, with a pageNumber column and a content column containing real, readable extracted text — not blank, not garbled.

Part 4 — Chunk the Extracted Text and Build a Chunk Identifier

Step 1: Start a new transform node

46. Select the Process Documents transform node.

47. Click the Transform button.

48. Rename the new transform node to Extract Chunks in the top left.

Step 2: Add the Chunk string block

49. Search for and add a Chunk string transform block.

50. Set the content column as the input expression.

51. Set Chunk size to 512 — smaller values give more specific chunks but longer execution time; 512 is a good balance for this use case, the same figure from Day 2's discussion of chunking strategy.

52. Set the output column name to content, replacing the existing column.

53. Click Apply.

Step 3: Add the Explode Array with Position block again

54. Search for and add another Explode Array with Position transform block.

55. Set the content column as the array to explode.

56. Set the output column name to content, replacing the existing column.

57. Click Apply.

Step 4: Add the Extract Many Struct Fields block again

58. Search for and add another Extract Many Struct Fields transform block.

59. Set the content column as the struct to process.

60. Set the first locator to position, extracting that property into a new column called chunkNumber.
61. Click Add item.
62. Set the second locator to element, extracting that property into the existing content column.
63. Click Apply, then Close.

Checkpoint — Confirm Before Continuing

Your short claim forms each produced at least one chunk, and the longer repair estimate and police report produced more than one — chunk size is approximate, so exact counts will vary.

Step 5: Build a unique chunk identifier

A chunk needs a stable, unique primary key before it can become an Ontology object — the same principle from Week 1, Part 1: design a real key, not a random one.

64. Select the Extract Chunks transform node.
65. Click the Transform button.
66. Rename the new transform node to Create Chunk ID in the top left.
67. Search for and add a Concatenate strings transform block.
68. Set the separator to an underscore (_).
69. Set the first element in the concatenation to the media_item_rid column — the unique identifier Foundry assigns to each uploaded file.
70. Set the second element to a Cast expression with type String applied to the pageNumber column.
71. Click Add item.
72. Set the third element to a Cast expression with type String applied to the chunkNumber column.
73. Name the new column chunkId.
74. Click Apply, then Close.

Takeaway

chunkId is built from media_item_rid, pageNumber, and chunkNumber specifically because that combination is guaranteed unique and traceable — you can look at any chunkId and immediately know which file, which page, and which chunk within that page it came from. This is exactly the same design discipline as choosing claimId or policyId as a primary key back in Week 1.

Checkpoint — Confirm Before Continuing

Every row in your preview now has a chunkId value with no duplicates, formatted as three parts joined by underscores.

Part 5 — Generate Embeddings for Each Chunk

Step 1: Start a new transform node

75. Select the Create Chunk ID transform node.
76. Click the Transform button.
77. Rename the new transform node to Embed Chunks in the top left.

Step 2: Add the Text to embeddings block

78. Search for and add a Text to embeddings transform block.

79. Set content as the column to embed.
80. Next to Model type, click Select a model.
81. Select text-embedding-ada-002 from the recommended models.
82. Set the output column name to embedding.
83. Click Apply, then Close.

Takeaway

The Text to embeddings block creates a vector representation of each chunk's meaning. Chunks covering similar topics end up with embeddings that sit close together in that vector space — this is what makes semantic search possible in Part 8, rather than only exact keyword matching.

Step 3: Create your output dataset

84. Select the Embed Chunks transform node.
85. Click the Add output button.
86. Click the New dataset button.
87. Rename the output dataset to ClaimDocumentChunks in the configuration panel on the right.

Step 4: Build and deploy your pipeline

88. Click the Deploy Pipeline icon in the right-side bar.
89. Click View deployment history in the panel that opens.
90. Click Build report in the row corresponding to the current build, and wait for your dataset to finish building — this typically takes a few minutes.

Checkpoint — Confirm Before Continuing

Your ClaimDocumentChunks dataset exists and has finished building, with one row per chunk, each containing chunkId, content, and a populated embedding vector column — not blank or null.

Part 6 — Create Your Claim Document Chunk Object Type

This Part returns to Ontology Manager — the same tool from Weeks 1 and 3, now used to bring your embedded chunks into the Ontology.

Step 1: Create a new object type

91. Open Ontology Manager and your Claims Operations Ontology.
92. Click New in the top right corner.
93. Select Object type from the dropdown.

Step 2: Select the backing dataset

94. Under Datasource, select Use existing datasource.
95. Click Select datasource.
96. Search for and select your ClaimDocumentChunks dataset.
97. Click Next.

Step 3: Define metadata

98. Name your object Claim Document Chunk. The plural name should auto-populate.
99. Select an icon that helps describe the object — a document or fragment-style icon is a good fit.

100. Click Next.

Step 4: Create object properties

101. Select chunkId as the Primary key of the new object — this serves as each object's unique identifier.
102. Select content as the Title — this is what displays across the platform when this object is shown in a list.
103. Click Next.

Platform Mechanic

Don't worry if there's a warning next to the embedding property at this point. Embedding is an advanced property type, and the warning simply flags that you may want to review its configuration later. For this lab, the default settings are sufficient — you do not need to manually configure anything about the vector itself.

Step 5: Create the object type

104. Click Create in the bottom right corner.

Step 6: Link it back to Claim

105. From your new Claim Document Chunk object type's page, create a new Link Type to Claim, using a shared claimId column on each side — the same Link Type process from Week 1, Part 6.

Common Mistake

Your ClaimDocumentChunks dataset needs a claimId column for this link to work. If you don't have one yet, return to Pipeline Builder and add it — for example, by extracting it from your original file naming convention, or by joining against a lookup of which file belongs to which claim, before your Part 5 output. This is worth confirming before you continue.

106. Save.

Checkpoint — Confirm Before Continuing

Your Claim Document Chunk object type exists, shows real chunk data when previewed, and is linked to Claim.

Part 7 — Search Your Chunks by Content

Before wiring this into your Copilot, confirm you can find a chunk by its content directly.

Step 1: Preview your object type's data

107. In Ontology Manager, open your Claim Document Chunk object type's data preview.
108. Manually search or filter for a specific word you know is in one of the provided documents — try “structural” (present in both CLM013's claim form and its repair estimate) or “pry marks” (present only in CLM064's police report).
109. Confirm the matching chunk (and only the matching chunk) appears.

Checkpoint — Confirm Before Continuing

You can find a specific chunk by searching for text you know is in it, confirming your extraction, chunking, embedding, and linking all worked correctly before moving to Part 8.

Part 8 — Add a Semantic Search Step to Your ClaimsCopilot

This Part uses AIP Logic's Semantic search function block — a dedicated block for searching an object set by meaning, using the embedding property you built in Part 5. This is different from, and more powerful than, the Query tool you used in Week 2: instead of matching exact values, it finds chunks whose meaning is closest to what you're looking for.

Step 1: Open your ClaimsCopilot function

110. In AIP Logic, open your published Week 2 ClaimsCopilot function — or, to keep your original safely intact, use Save As to create a new version named ClaimsCopilotV2.

Step 2: Scope your chunks to this claim

You want to search only the chunks belonging to the claim currently being processed, not every chunk in your Ontology.

111. From the blocks toolbar, click the Ontology dropdown and add a block that retrieves Claim Document Chunk objects filtered to where claimId equals this function's claimId input — producing a filtered object set variable.
112. Name this variable's output something explicit, such as thisClaimsChunks, if the block allows you to.

Step 3: Add the Semantic search block

113. From the blocks toolbar, click the magnifying glass icon to add a Semantic search function block.
114. Click the Object set dropdown.
115. Instead of Create new object set, select your Step 2 filtered variable (thisClaimsChunks) so the search is scoped to this claim only.
116. Select Embedding as the Property to search.
117. Set the Number of returned objects to 5.
118. Set the Query to a fixed, descriptive phrase relevant to claims review, such as “cause of loss, damage description, and any evidence of inconsistency.”

Platform Mechanic

A new object set variable, following the naming pattern Searched All (...), is created automatically by this block. You'll reference it directly in your Use LLM block's prompt in the next step.

Step 4: Update your Use LLM block

119. Open your existing Use LLM function block — the same block from Week 2 that produces your Copilot's structured output.
120. In the Instructions (System prompt) field, add the following, without deleting your existing instructions:

Instructions (System prompt) Addition — Part 8

```
You now have access to real extracted text from this claim's documents, retrieved by semantic search below. When answering questions about what a document says, quote or closely paraphrase what the retrieved chunks actually contain, rather than inferring from the Claim object's structured fields alone. If the retrieved chunks don't contain relevant information, say so explicitly — do not guess at document content that wasn't retrieved.
```

121. In the Provide input data (Task prompt) field, reference your semantic search results: type “/”, select your Searched All (...) variable from Step 3, then select the content property from the property picker that appears, and click Add 1 property. This inserts a formatted variable containing the retrieved chunk text directly into your prompt.

Checkpoint — Confirm Before Continuing

Running your updated ClaimsCopilot against CLM013 now references the actual wording from its claim form and repair estimate — for example, the specific structural concern — not just structured field values.

Part 9 — Test and Verify End to End

122. Run your updated ClaimsCopilot against CLM014. Confirm its response reflects the actual content of the auto claim form — a rear-end collision with the other driver at fault.
123. Run it against CLM013. Confirm it picks up the escalation-relevant language from both documents — the fire department's structural concern and the repair estimate's structural line item.
124. Run it against CLM064. Confirm it surfaces the genuine discrepancy between the two documents — the claim form's forced-entry account against the police report's finding of no physical evidence of forced entry — rather than smoothing over the inconsistency.
125. Run it against a claimId with no uploaded documents at all, such as CLM001. Confirm it explicitly states no document content was found, rather than fabricating an answer.

Checkpoint — Confirm Before Continuing

You have tested and can show evidence for all four cases: real content correctly used for CLM014, escalation-relevant content correctly surfaced for CLM013, the genuine discrepancy correctly surfaced for CLM064, and an honest “nothing found” response when no chunks exist.

Part 10 — Create an Evaluation Suite and Publish

126. After a successful Preview run, look for Add as test case near your run results.
127. Add at least four test cases: CLM014 (straightforward), CLM013 (High-Value, escalation language), CLM064 (the forced-entry discrepancy), and a claimId with no documents at all, such as CLM001.
128. Publish your final version.

Checkpoint — Confirm Before Continuing

An evaluation suite exists with at least four test cases, and your final version is published.

Full Validation Checklist

- ☐ All five provided PDF documents confirmed and opened, matched to their real claimId values (CLM014, CLM013, CLM064)
- ☐ ClaimDocuments media set created, containing all uploaded files
- ☐ Extract text from PDF, Explode Array with Position, Extract Many Struct Fields, and Drop Columns blocks produce one clean row per page
- ☐ Chunk string, Explode Array with Position, and Extract Many Struct Fields blocks produce multiple chunks per longer document
- ☐ Concatenate strings block produces a unique chunkId for every chunk
- ☐ Text to embeddings block produces a populated vector column using text-embedding-ada-002
- ☐ Claim Document Chunk object type created, backed by ClaimDocumentChunks, and linked to Claim
- ☐ A specific chunk can be found by searching for known content

- ☐ ClaimsCopilot (or ClaimsCopilotV2) uses a Semantic search block scoped to the current claim, and grounds its answers in retrieved chunk content
- ☐ All four test cases verified: CLM014 content used, CLM013 escalation content surfaced, CLM064 discrepancy surfaced, honest no-content response
- ☐ Evaluation suite has 4+ test cases; final version published

Reflection Questions

129. chunkId is built from media_item_rid, pageNumber, and chunkNumber rather than a random or generated value. What would go wrong later in this pipeline if two different chunks ended up with the same chunkId?
130. Your Claim Document Chunk object type is linked to Claim the same way your Week 1 Payment or Investigation objects are. What does that tell you about how Foundry treats a piece of a document compared to any other kind of record?
131. CLM064's claim form and police report don't actually agree with each other. What would your ClaimsCopilot need to do differently if it only ever read one of the two documents, instead of both?
132. Part 8's Semantic search block found chunks by meaning, using a fixed query phrase, rather than an exact keyword match like Part 7. What kind of claims-handling question would this approach answer well, and what kind would it likely miss?

If You Run Out of Time

Complete Parts 1–6 (a working pipeline through to a linked, embedded Claim Document Chunk object type) as the minimum before Day 4. Parts 7–10 (search, Copilot integration, testing, and publishing) can be finished as the first thing you do on Day 4 if needed.

Troubleshooting

Symptom	Likely Cause & Fix
Extract text from PDF returns blank text	Confirm you selected the correct mediaReference column in the Media reference dropdown, and that your media set actually uploaded successfully in Part 2
Extract text from PDF returns garbled or partial text	This can happen if a document was actually scanned rather than born-digital — try switching Extract Method to OCR for that specific file
Explode Array with Position produces unexpected row counts	Confirm you selected the correct array column (content) as the array to explode — a wrong column selection will explode the wrong data
Chunk String block produces only one chunk for a longer document	Confirm Chunk size is set to 512, not left at a default that's larger than your document
Concatenate strings block fails to apply	Confirm both Cast expressions are set to type String — concatenating a number directly without casting it first will cause an error
Text to embeddings isn't available in the block search	Confirm you're searching from the transform block search bar, not a different menu — search for “embed” if “Text to embeddings” doesn't appear first
Claim Document Chunk doesn't show up as linkable to Claim	Confirm your ClaimDocumentChunks dataset actually has a claimId column with matching values and types — the same Link Type requirement from Week 1, Part 6

Symptom	Likely Cause & Fix
Semantic search block's Object set dropdown only shows "Create new object set"	Confirm you completed Part 8, Step 2 first — the filtered variable must exist in your function before it can be selected here
ClaimsCopilot doesn't reference document content after Part 8	Confirm your Task prompt actually includes the added property (via Add 1 property) from your Searched All (...) variable, and that your Instructions addition was saved
Can't find Pipeline Builder, Ontology Manager, or AIP Logic at all	Use Cmd+J (macOS) or Ctrl+J (Windows) for quick search from anywhere in Foundry, and type the application name directly